Friday 10 January 2025

- added an optional reading page to Quercus.
  - really is optional - look if interested.

So far ... we have done two examples that show A Math Textbook is not Enough if your goal is to compute accurate results.

Last example, trying to compute $y_n = \int_0^1 \frac{x^n}{x+5}\,dx$ for $n = 0, 1, 2, \ldots, 8$ to 3 decimal digits.

Applying the <u>exact</u> recurrence $y_{n+1} = \frac{1}{n+1} - 5y_n$ with 3 digit arithmetic quickly lead to inaccurate results as the effect on any small error E was magnified by 5 each iteration.

How can we modify our work to get accurate results?

Rearrange the exact recurrence:

$$y_{n+1} = \frac{1}{n+1} - 5y_n$$

$$5y_n = \frac{1}{n+1} - y_{n+1}$$

or $$y_n = \frac{1}{5}\left(\frac{1}{n+1} - y_{n+1}\right)$$

We can compute the sequence backwards.

How to start? We know

$$y_{100} < y_{99} < y_{98} < \cdots < y_0 \doteq 0.182$$

Let's approximate $y_{100} \doteq 0$.

We know $y_{100} = 0 + \varepsilon$, for some $0 < \varepsilon < 0.182$.

then
$$y_{99} = \frac{1}{5}\left(\frac{1}{100} - y_{100}\right)$$
$$= \frac{1}{5}\left(\frac{1}{100} - (0+\varepsilon)\right)$$
$$= \frac{1}{500} - \varepsilon/5.$$

then
$$y_{98} = \frac{1}{5}\left(\frac{1}{99} - \left(\frac{1}{500} - \varepsilon/5\right)\right)$$
$$= \text{---} + \varepsilon/5^2$$

Observe that we make the effect of our original error ε smaller on each iteration. Eventually the effect of this initial error will be insignificant. Also, any other error introduced along the way will also be reduced each iteration.

By the time we get to $y_8, y_7, \ldots, y_0$ we will have accurate results!

The initial error in our forwards approach came from approximating $\ln(6/5)$ by 0.182.

This suggests that it will be important for us to understand the accuracy of computer numbers and arithmetic, which is our next topic.

Floating Point Numbers and Floating Point Arithmetic

There are an uncountable number of real numbers. In addition, irrational numbers like π or e require an infinite number of digits to represent.

Since computer memory is finite, we need to make approximations and are only able to exactly store a finite number of values.

Computers approximate real numbers by floating-point numbers with the form:

$$\pm \boxed{d_0 . d_1 d_2 \cdots d_{p-1}} \times \beta^{E}$$

sign — point — significand or mantissa — exponent

where β : base of the floating point system

$0 \leq d_i < \beta$ are the significant digits

p : is the precision of the f.p. system
- specifies number of digits carried
$\therefore$ controls the accuracy

E : is the exponent.

we have $L \leq E \leq U$

control the range of the numbers

This gives a finite set $F(\beta, p, L, U)$ of floating point numbers.

One issue: representation is not unique

e.g. with $\beta = 10$, $p = 5$, $L = -10$, $U = 10$

$$
\begin{aligned}
& 1.2300 \times 10^{0} && \in F(10, 5, -10, 10) \\
= \; & 0.1230 \times 10^{1} && \in F(10, 5, -10, 10) \\
= \; & 0.0123 \times 10^{2} && \in F(10, 5, -10, 10)
\end{aligned}
$$

Computer scientists prefer unique representations for quantities and make rules to avoid ambiguity / non-uniqueness.

enforce rule: $d_0 \neq 0$

This normalizes the floating-point system and makes representations unique.

But ... how do you then represent 0 ?!

Make a new rule: set $E = L - 1$

signals a denormalized number

and use $0.00\ldots0 \times \beta^{L-1}$ to represent 0.

In the early days of computing different computer manufactures used different values for

β, E, L, U e.g $\beta = 2, 3, 8, 10, 16$

DEC, CDC, IBM 7090 — Setun — Burroughs — IBM 360, IBM 5360

This made writing portable software more difficult as operating conditions changed. It locked people into using a particular brand.

The variation was not desirable, and IEEE Standards were produced (1980's).

The standard defines.

	single precision	double precision
β	2	2
p	24	53
L	-126	-1022
U	+127	+1023

$-126, -125, \ldots, 0, \ldots, 127$
total of 254 values
use 8 bits
Leaves room for $E = L-1$ denormalized
$E = U+1$ Inf, NaN

memory requirement 32 bits → 64 bits
this is what "doubles"

count: 1 + 8 + 24 = 33 bits
sign exponent significand

Since $0 < d_0 < 2$ when normalized ??
we know $d_0 = 1$
and so we do not need to store it!

C: float double

python: — float

for accurate engineering work,
extended precision (quad) is used.

in machine learning, have more data and need less accuracy:

Google Brain bfloat16

±	exponent	significand
1	8	7

vs IEEE single

±	exponent	significand
1	8	23

$p=24$

Current 2025! proposals for ML
IEEE P3109

binary8p3

±	exp	sign.
1	5	2

$p=3$ since $d_0=1$.

To Be Continued on Monday